

Project: Patient Appointment Management System

1. Overview

The **Patient Appointment Management System** is a web-based application designed to streamline the process of booking, managing, and tracking appointments in a healthcare facility. It enables patients to schedule appointments, doctors to manage their availability, and administrators to oversee the system.

The system adheres to the **MVC architecture** and is compatible with **Spring MVC (Java)** and **ASP.NET Core MVC (.NET)** frameworks. It focuses on improving patient care by reducing scheduling conflicts and ensuring efficient communication between patients and healthcare providers.

2. Assumptions

- Supports multiple user roles: Patient, Doctor, and Administrator.
- Doctors can specify their availability, and patients can book appointments based on open slots.
- Notifications are sent for appointment confirmation and reminders.
- Uses MySQL or SQL Server as the database.

3. Module-Level Design

3.1 Patient Management Module

Purpose: Manages patient registration, profiles, and appointment history.

- **Controller:**
 - PatientController
 - registerPatient()
 - loginPatient()
 - updatePatientProfile()
 - viewAppointmentHistory()
- **Service:**
 - PatientService
 - Handles business logic for patient registration, authentication, and profile updates.
- **Model:**
 - **Patient Entity**
 - **Attributes:**
 - patientId (PK)
 - name
 - email
 - phone
 - address
 - dob (Date of Birth)

3.2 Doctor Management Module

Purpose: Manages doctor profiles, availability, and schedules.

- **Controller:**
 - DoctorController
 - addDoctor()
 - updateDoctorAvailability()
 - viewDoctorSchedule()
- **Service:**
 - DoctorService
 - Implements business logic for managing doctor profiles and availability.
- **Model:**
 - **Doctor Entity**
 - Attributes:
 - doctorId (PK)
 - name
 - specialization
 - email
 - phone
 - availability (Start Time, End Time)

3.3 Appointment Management Module

Purpose: Enables patients to book, modify, and cancel appointments.

- **Controller:**
 - AppointmentController
 - bookAppointment()
 - cancelAppointment()
 - viewAppointments()
- **Service:**
 - AppointmentService
 - Implements logic for booking and managing appointments.
- **Model:**
 - **Appointment Entity**
 - Attributes:
 - appointmentId (PK)
 - patientId (FK to Patient)
 - doctorId (FK to Doctor)
 - appointmentDate
 - timeSlot
 - status (BOOKED, CANCELED, COMPLETED)

3.4 Admin Management Module

Purpose: Oversees system operations, including managing user roles and resolving scheduling conflicts.

- **Controller:**
 - AdminController

- addAdmin()
 - viewSystemLogs()
 - manageUsers()
- **Service:**
 - AdminService
 - Handles administrative tasks such as user management.
- **Model:**
 - **Admin Entity**
 - Attributes:
 - adminId (PK)
 - name
 - email
 - role

3.5 Notification Module

Purpose: Sends notifications for appointment confirmations, cancellations, and reminders.

- **Controller:**
 - NotificationController
 - sendAppointmentReminder()
 - sendConfirmationNotification()
- **Service:**
 - NotificationService
 - Implements email/SMS notification logic.
- **Model:**
 - **Notification Entity**
 - Attributes:
 - notificationId (PK)
 - recipientId
 - message
 - notificationType
 - sentDate

4. Database Schema

4.1 Table Definitions

1. Patient Table

```
CREATE TABLE Patient (
    patientId INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(15),
    address TEXT,
    dob DATE
);
```

2. Doctor Table

```
CREATE TABLE Doctor (  
    doctorId INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    specialization VARCHAR(100),  
    email VARCHAR(100),  
    phone VARCHAR(15),  
    availability JSON  
);
```

3. Appointment Table

```
CREATE TABLE Appointment (  
    appointmentId INT AUTO_INCREMENT PRIMARY KEY,  
    patientId INT,  
    doctorId INT,  
    appointmentDate DATE,  
    timeSlot TIME,  
    status ENUM('BOOKED', 'CANCELED', 'COMPLETED'),  
    FOREIGN KEY (patientId) REFERENCES Patient(patientId),  
    FOREIGN KEY (doctorId) REFERENCES Doctor(doctorId)  
);
```

4. Admin Table

```
CREATE TABLE Admin (  
    adminId INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    role VARCHAR(50)  
);
```

5. Notification Table

```
CREATE TABLE Notification (  
    notificationId INT AUTO_INCREMENT PRIMARY KEY,  
    recipientId INT,  
    message TEXT,  
    notificationType ENUM('CONFIRMATION', 'REMINDER'),  
    sentDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5. Local Deployment Details

1. Environment Setup:

- Install MySQL or SQL Server and create the database using the schema provided.
- Configure the application server (Tomcat for Spring MVC or IIS/Kestrel for ASP.NET Core).
- Set up application properties for database connection (e.g., application.properties or appsettings.json).

2. Deployment Steps:

- Clone the repository and build the project using Maven or Visual Studio.
- Run the application locally using the appropriate server configuration.
- Access the application at `http://localhost:<port_number>`.

6. Conclusion

The **Patient Appointment Management System** enhances the efficiency of healthcare operations by providing a seamless platform for appointment scheduling, patient management, and notifications. It ensures a better experience for patients and healthcare professionals while maintaining system reliability and scalability.